

ZYVORAI LABS · CUSTOMER DOCUMENTATION

Chimera

Getting Started

2026-09-04

What you need

- Go 1.25+ to build from source, **or** a `.deb` / `.rpm` from [GitHub Releases](#)
- Browser for Command Center at `/__chimera/`
- Optional: Transiva checkout for end-to-end export smoke

1. Start the engine

```
git clone https://github.com/zyvora/chimera.git
cd chimera
go mod tidy
go build -o bin/chimera ./cmd/chimera
./bin/chimera serve -config config.example.json
```

Or remote install:

```
./scripts/deploy-remote.sh <host> [user]
```

2. Open Command Center

```
http://<host>:8080/chimera/
```

Log in with **admin** / **admin** (change later in Settings). Without a session the UI shows only the login screen — no inventory flash.

3. Confirm the vSphere persona

```
./bin/chimera selftest  
-url http://<host>:8989/sdk \\  
-user administrator@vsphere.local \\  
-pass vmware
```

A passing self-test proves auth, inventory, OVF descriptor, `ExportVm`, and NFC lease download.

Optional: protocol personas (Nutanix, Hyper-V, AWS, Azure)

Command Center stays on the vSphere listener. For other protocol smoke tests, start a second process (or replace the persona) with env overrides:

```
CHIMERA_PERSONA=nutanix CHIMERA_USERNAME=admin CHIMERA_PASSWORD=secret \  
./bin/chimera serve -listen 0.0.0.0:8990  
→ http://<host>:8990/api/nutanix/v3  
  
CHIMERA_PERSONA=hyperv CHIMERA_USERNAME=Administrator CHIMERA_PASSWORD=secret \  
./bin/chimera serve -listen 0.0.0.0:8991  
→ http://<host>:8991/wsman  
  
CHIMERA_PERSONA=aws CHIMERA_USERNAME=AKIDCHIMERA CHIMERA_PASSWORD=chimera-secret \  
./bin/chimera serve -listen 0.0.0.0:8992  
→ EC2 Query at http://<host>:8992  
  
CHIMERA_PERSONA=azure CHIMERA_USERNAME=11111111-2222-3333-4444-555555555555 \  
CHIMERA_PASSWORD=chimera-azure-token ./bin/chimera serve -listen 0.0.0.0:8993  
→ ARM at http://<host>:8993/subscriptions/...
```

See product-repo `README.md`, `docs/PROVIDER_ARCHITECTURE.md`, and `docs/AWS_AZURE_PERSONAS.md`.

4. First useful checks in the UI

1. Overview KPIs populate after self-test (requests, sessions, exports).
2. Topology shows datacenter → cluster → host → datastore.
3. VM inventory lists simulator VMs (for example `DC0_C0_RP0_VM0`).
4. Scenario Launcher: try **Clean Environment**, then **Slow Fabric** and watch latency in Live requests.

Operate from the console (UX)

1. Browse to `http(s)://<host>:8989/__chimera/` and sign in (**admin / admin** unless rotated).
2. Confirm sidebar **System Health** ring and footer **vSphere Persona** strip show uptime and SDK endpoint.
3. Click  **Refresh** on Overview if KPIs are flat, then run `selftest` from the host.
4. Open **Inventory** — verify VM count badge matches config; copy a VM name with the row  action.
5. Optional: **Settings** (gear) → change admin password before sharing the lab URL.

Empty / fail: Login loop → defaults or `CHIMERA_ADMIN_*` ; KPIs zero after selftest → client must use `/sdk` URL.
Inventory empty → restart after config estate changes.

Success: KPIs and Live requests move during selftest; topology subtitle shows your datacenter/cluster counts.

Next

- [Using the Dashboard](#)
 - [Wire Transiva](#)
 - [Admin basics](#)
 - [Page-by-page guides](#)
-

Chimera's **Command Center** is a single-page infrastructure console embedded in the Go binary (`internal/gateway/ui.go`). There is no Node/React build step — open `/__chimera/` on the same listener as the vSphere SDK.

Shell layout

Region	Role
Left rail	Control Center nav, provider personas (vSphere active in UI; Nutanix/Hyper-V/AWS/Azure via CHIMERA_PERSONA ; others roadmap), System Health ring
Top bar	Global search (🔍), Scenario shortcut, Fault Studio (bolt), notifications bell → Live requests, Settings (gear), user menu (logout)
Main column	Overview KPIs, topology, Top Activity, Live requests, VM inventory, Scenario Launcher, VMDK Library
Right drawers	Fault Studio (fault injection) and Settings (listen address + admin credentials)

How to move around

1. Sign in — mutating controls (scenarios, faults, VMDK upload, credential save) require a dashboard session.
2. Use sidebar anchors or scroll — Overview is one long workbench page with hash targets (`#topology` , `#inventory` , `#vmdks` , `#requests` , `#activity` , `#scenarios`).
3. Click  **Refresh** after scenario or fault changes if you want an immediate telemetry pull (otherwise ~2s polling).
4. Open **View Full Topology** for the zoomable estate map (**Fit / +** , Escape to exit).
5. Filter VMs by name or power state; paginate 5 per page; global search focuses inventory.
6. Open **Fault Studio** from the bolt icon (not the gear — that is Settings).

Operate from the console (UX)

- 1. Land on Overview after login; read six KPI tiles and sidebar health ring.
- 2. Drive traffic with `selftest` or Transiva against `http(s)://<host>:8989/sdk` — dashboard polls do not inflate metrics.
- 3. Use **Scenario Launcher** for presets; open **Fault Studio** for manual latency/failure/NFC drop tuning.
- 4. Stage real disks via **VMDK Library** when testing conversion, not just synthetic export.
- 5. Bell icon jumps to **Live requests** when errors spike; toggle **View All** for 30-row buffer.
- 6. Rotate admin password under **Settings** before exposing `<host>` beyond a lab VLAN.

Empty / fail: See table below; page guides under [pages/](#) have surface-specific checks.

Symptom	Check
Login loop	Defaults <code>admin / admin</code> ; or values set via Settings / <code>CHIMERA_ADMIN_*</code>
Empty inventory	Simulator counts in config; restart after config changes
No KPI movement	Clients must hit <code>/sdk</code> , not <code>/__chimera/</code>
Self-test fails	URL must be the SDK (<code>/sdk</code>), not Command Center
Fault Studio vs Settings	Bolt = faults; gear = settings

Success: KPIs, activity donut, and live request feed update during exports; scenarios toast confirmation; VM/fixture badges reflect VMDK matching.

Next: [Page-by-page guides](#) · [Workflows](#)

End-to-end jobs mixing CLI and Command Center. Use `<host>` for any lab address — never hard-code deployment IPs in runbooks.

A. Prove the vSphere contract (self-test)

```
./bin/chimera selftest  
-url http://<host>:8989/sdk \\  
-user administrator@vsphere.local \\  
-pass vmware
```

Expect: login → inventory → OVF descriptor → ExportVm → NFC download success.

Operate from the console (UX)

1. Sign in at `http(s)://<host>:8989/__chimera/`.
2. Run selftest from the host (command above).
3. Watch **Requests**, **Exports**, and **Avg Response** KPIs tick up.
4. Open **Live requests** — confirm SOAP/SDK/NFC paths with 2xx statuses.
5. **Top Activity** donut should gain segments after the run.

Fail: KPIs flat → selftest URL wrong (must be `/sdk`). Red statuses → [Fault Studio](#) → **Reset** or **Clean Environment** scenario.

B. Wire Transiva export

```
/scripts/make-transiva-config.sh > /tmp/transiva-chimera.yaml  
Point Transiva / hyperexport at that config (see docs/TRANSIVA.md)
```

Confirm `vcenter_url` uses Chimera's `<host>` (not laptop `127.0.0.1` if Transiva runs elsewhere) and `insecure: true` for lab TLS.

Operate from the console (UX)

1. Copy a target VM from **Inventory** (row ↓ or **Export VM** after filtering).
2. Start Transiva export; keep Command Center open on **Live requests**.
3. Monitor **Exports** KPI and NFC rows; check **Fixture** badge (`Synthetic` vs real VMDK).
4. Use footer SDK endpoint string when verifying Transiva points at the same `<host>` .

Success: Steady NFC traffic, rising byte counter, green statuses unless faults armed.

C. Resume / Range download

```
./scripts/scenario.sh resume  
or click Resume Export in Scenario Launcher
```

Drops the next NFC stream after ~2 MiB so Transiva retries with `Range: bytes=N-` and Chimera returns HTTP 206.

Operate from the console (UX)

1. Sign in → **Scenario Launcher** → **Resume Export** (or CLI above).
2. Run export; watch **Live requests** for initial failure then **206** on retry.
3. **Clean Environment** or Fault Studio **Reset** before the next baseline run.

Fail: No 206 → client may not send Range; see product `docs/TRANSIVA.md` .

D. Fault injection for client hardening

Operate from the console (UX)

1. Open **Fault Studio** (bolt icon or sidebar).
2. Set latency, **Fail next requests**, NFC drop fields, or bandwidth cap → **Apply policy**.
3. Re-run Transiva export or `selftest` ; correlate with red rows in **Live requests**.
4. **Reset** or **Clean Environment** before benchmark runs.

Success: Injected errors visible in feed and error-rate KPI; clean reset returns nominal metrics.

E. Real VMDK fixtures

1. Stage `.vmdk` files under `fixture_vmdk_dir` or use **VMDK Library** → **Upload VMDK**.
2. Optionally pin a file to a VM (**Manual** fixture badge in Inventory).
3. Export that VM through Transiva to exercise conversion, not only transfer.

Operate from the console (UX)

1. **VMDK Library** → confirm file listed with assignment method.
2. **Inventory** → hover fixture badge for filename; expect **Matched** / **Manual** instead of **Synthetic**.
3. Run export; verify larger disk payload in telemetry bytes transferred.

Matching rules: see product-repo `docs/TRANSIVA.md` .

Related pages

- [Using the Dashboard](#)
- [Admin basics](#)
- [Page-by-page guides](#)